Implement an 8-puzzle problem using
i. DFS
ii. Iterative deepening search (IDS) algorithm

```python
# 8-Puzzle using DFS

initial = (1, 2, 3,
           4, 5, 6,
           7, 0, 8)

goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)


def get_neighbors(state):
    neighbors = []

    blank = state.index(0)
    row, col = divmod(blank, 3)

    moves = [(-1, 0), (1, 0),
             (0, -1), (0, 1)]

    for dr, dc in moves:
        r = row + dr
        c = col + dc

        if 0 <= r < 3 and 0 <= c < 3:
            pos = r * 3 + c
            board = list(state)

            board[blank], board[pos] = (
                board[pos], board[blank]
            )

            neighbors.append(tuple(board))

    return neighbors
```

```python
def dfs(state, goal, visited, path, limit):
    if state == goal:
        return path

    if limit == 0:
        return None

    for next_state in get_neighbors(state):
        if next_state not in visited:
            visited.add(next_state)

            result = dfs(
                next_state,
                goal,
                visited,
                path + [next_state],
                limit - 1
            )

            if result is not None:
                return result

            visited.remove(next_state)

    return None


def print_solution(solution):
    if solution is None:
        print("No solution found within the depth limit.")
        return

    print("Solution found!")
    print("Moves:", len(solution) - 1)

    for step, state in enumerate(solution):
        print("Step", step)
        for i in range(0, 9, 3):
            print(state[i:i+3])
        print()


solution = dfs(initial, goal, {initial}, [initial], 20)
print_solution(solution)
```

```
Solution found!
Moves: 19
Step 0
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

Step 1
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

Step 2
(1, 0, 3)
(4, 2, 6)
(7, 5, 8)

Step 3
(0, 1, 3)
(4, 2, 6)
(7, 5, 8)

Step 4
(4, 1, 3)
(0, 2, 6)
(7, 5, 8)

Step 5
(4, 1, 3)
(2, 0, 6)
(7, 5, 8)

Step 6
(4, 0, 3)
(2, 1, 6)
(7, 5, 8)

Step 7
(0, 4, 3)
(2, 1, 6)
(7, 5, 8)

Step 8
(2, 4, 3)
(0, 1, 6)
(7, 5, 8)

Step 9
(2, 4, 3)
(1, 0, 6)
(7, 5, 8)
```

```
Step 10
(2, 4, 3)
(1, 5, 6)
(7, 0, 8)

Step 11
(2, 4, 3)
(1, 5, 6)
(7, 8, 0)

Step 12
(2, 4, 3)
(1, 5, 0)
(7, 8, 6)

Step 13
(2, 4, 3)
(1, 0, 5)
(7, 8, 6)

Step 14
(2, 0, 3)
(1, 4, 5)
(7, 8, 6)

Step 15
(0, 2, 3)
(1, 4, 5)
(7, 8, 6)

Step 16
(1, 2, 3)
(0, 4, 5)
(7, 8, 6)

Step 17
(1, 2, 3)
(4, 0, 5)
(7, 8, 6)

Step 18
(1, 2, 3)
(4, 5, 0)
(7, 8, 6)

Step 19
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

ii. Using Iterative deepening search (IDS) algorithm

OUTPUT:

```
Depth limit: 0
Depth limit: 1
Solution found!
Moves: 1
Step 0
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

Step 1
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```